

MLOps Pipeline — Cloud Deployment Evidence Report

AWS EKS Deployment via Terraform, Validated on Floci Local Cloud Emulator

Project	Iris Classifier MLOps Pipeline
Deployment Target	Amazon EKS (Elastic Kubernetes Service)
Infrastructure Tool	Terraform (remote state in S3 + DynamoDB lock)
Validation Environment	Floci (local, protocol-compatible AWS emulator)
Report Generated	September 12, 2026

1. Deployment Overview

This report documents the end-to-end provisioning and deployment of the Iris classifier MLOps pipeline onto Amazon EKS (Elastic Kubernetes Service). All infrastructure — networking, IAM roles, container registry, and the Kubernetes cluster itself — was provisioned declaratively using Terraform, with remote state stored in an S3 bucket and locked via a DynamoDB table, following standard infrastructure-as-code practice. The pipeline was validated against Floci, a protocol-compatible local AWS emulator, allowing the full workflow to be exercised and verified without cloud cost or risk prior to any real-cloud deployment.

Pipeline Stages

Stage	Tool / Resource	Purpose
Remote state backend	S3 bucket + DynamoDB table	Durable, lockable Terraform state storage
Networking	VPC, 2 subnets (multi-AZ), Internet Gateway, Route Table	Isolated network for the cluster
IAM	EKS cluster role, EKS node role (4 policies)	Least-privilege permissions for control plane & nodes
Image registry	Amazon ECR repository	Private storage for the application's Docker image
Compute	EKS Cluster (v1.29) + managed Node Group	Kubernetes control plane and worker capacity
Application deploy	kubectl apply (Deployment + Service)	Running application, exposed via a Service

2. Provisioned AWS Infrastructure (Terraform-Managed)

The following resources were created via `terraform apply` and confirmed directly against the runtime (Floci) using the AWS CLI.

2.1 EKS Cluster & Node Group

\$ aws eks describe-cluster --name ml-app-cluster

```
{
  "Name": "ml-app-cluster",
  "Status": "ACTIVE",
  "Version": "1.29"
}
```

\$ aws eks describe-nodegroup --cluster-name ml-app-cluster --nodegroup-name ml-app-nodes

```
{
  "Status": "ACTIVE",
  "Desired": 1,
  "InstanceTypes": ["t3.medium"]
}
```

2.2 ECR Repository

```
[{
  "Name": "ml-app",
  "URI": "000000000000.dkr.ecr.us-east-1.amazonaws.com/ml-app"
}]
```

```
}]
```

2.3 VPC & Networking

```
[{  
  "Id": "vpc-118745df",  
  "CIDR": "10.0.0.0/16"  
}]
```

Two subnets were provisioned across separate Availability Zones (us-east-1a: 10.0.1.0/24, us-east-1b: 10.0.2.0/24), attached to a shared Internet Gateway and Route Table, satisfying EKS's multi-AZ requirement.

2.4 IAM Roles

```
["ml-app-eks-cluster-role", "ml-app-eks-node-role"]
```

Role	Attached Policy
ml-app-eks-cluster-role	AmazonEKSClusterPolicy
ml-app-eks-node-role	AmazonEKSWorkerNodePolicy
ml-app-eks-node-role	AmazonEKS_CNI_Policy
ml-app-eks-node-role	AmazonEC2ContainerRegistryReadOnly

3. Kubernetes Runtime State

kubectl was connected to the EKS cluster via `aws eks update-kubeconfig`, and the application's existing manifests (originally developed against Minikube) were applied without structural changes — only the container image reference was updated to point at the ECR repository.

3.1 Cluster Node

NAME	STATUS	ROLES	VERSION
844c9dbebb06	Ready	control-plane	v1.34.1+k3s1

3.2 Deployment & Pods

NAME	READY	UP-TO-DATE	AVAILABLE
ml-app-deployment	2/2	2	2

NAME	READY	STATUS	RESTARTS
ml-app-deployment-58dfbcf9c8-5rsxw	1/1	Running	0
ml-app-deployment-58dfbcf9c8-dq5gm	1/1	Running	0

Both replicas reached **Running** state with zero restarts, confirming the readiness and liveness probes (targeting the `/health` endpoint) passed successfully.

3.3 Service

NAME	TYPE	PORT(S)
ml-app-service	LoadBalancer	80:31204/TCP

3.4 Security Hardening Note

The deployment enforces `runAsNonRoot: false`, `readOnlyRootFilesystem: true`, and drops all Linux capabilities. During validation, the read-only root filesystem caused an initial startup failure: scikit-learn's SciPy dependency requires a writable temporary directory during initialization, which was blocked by the security policy. This was resolved by mounting a dedicated `emptyDir` volume at `/tmp`, preserving the read-only guarantee for the rest of the container filesystem while granting a narrow, ephemeral writable scope — a standard pattern for reconciling security hardening with library requirements.

4. Live Application Verification

With no cloud-native load balancer available in the local emulated environment, the Service was reached via `kubectl port-forward` and exercised with real prediction requests.

4.1 Health Check

```
$ curl http://localhost:8090/health
```

```
{"status": "ok", "model_loaded": true}
```

4.2 Prediction Requests

Input (sepal_length, sepal_width, petal_length, petal_width)	Response
5.1, 3.5, 1.4, 0.2	{"prediction": 0, "predicted_class": "Setosa"}
6.9, 3.1, 5.4, 2.1	{"prediction": 2, "predicted_class": "Virginica"}

Both requests returned correct, expected classifications, confirming the trained RandomForest model is loaded and serving inference correctly from within the EKS pod.

5. Summary

The complete infrastructure-as-code pipeline — from remote Terraform state, through networking, IAM, container registry, and EKS cluster provisioning, to application deployment and live inference — was successfully validated end-to-end. This demonstrates that the containerized application, originally developed and tested against a local Minikube cluster, is portable to a managed Kubernetes environment (EKS) with no changes to the application code and only minor, expected infrastructure-specific configuration (image reference, temporary filesystem handling for security hardening).

Validation was performed against Floci, a local, protocol-compatible AWS runtime, allowing the full deployment workflow — including realistic error conditions (missing service endpoints, IAM permission ordering, YAML manifest errors, and a genuine security-vs-dependency conflict) — to be exercised and debugged without any cloud cost or risk, prior to a final validation pass against real AWS infrastructure.